Final Year Project Technical Report

BSc (Hons) Computing with Computer Games Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

How effectively can Bluetooth and GPS work together to provide seamless location tagging?

Student Name: Dale Nesbitt

Student ID: L00106269

Declaration

1) I have accurately identified and included the sources of all facts, ideas, opinions, and viewpoints from others in the assignment references. All direct quotations, paraphrasing, and discussions of ideas from books, journal articles, internet sources, course materials, or any other sources used are properly acknowledged and cited in the assignment references.
2) I have not used unauthorised artificial intelligence tools or aids.
3) I understand and am compliant with ATU's policy and procedures regarding Academic Integrity and I am aware of the consequences of any violations.
4) I have followed the referencing guidelines recommended in the assignment instructions and / or programme documentation.
5) I understand that not adhering to the word limits on each section will result in penalties.
6) By signing this form or by submitting material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other programme of study.
7) By signing this form or by submitting material for assessment online I confirm that I have read and understood AQAE022 Academic Integrity Policy.

I confirm I agree to this declaration. ☒

Print Signature: Dale Nesbitt

Date: 21/04/2026

Acknowledgements

I would like to thank…

Table of Contents

1. Project Overview 1
   1.1. Purpose 1
   1.2. Objectives 1
      1.2.1 BLE Proximity Detection 1
      1.2.2 GPS Location Detection/Acquisition 1
      1.2.3 BLE/GPS Handshake 1
      1.2.4 Event Logging 2
      1.2.5 System Feedback 2
      1.2.6 Performance Evaluation 2
   1.3. Project Scope 2
2. Requirements 4
   2.1. Functional Requirements 4
   2.2. Non-Functional Requirements 5
3. System Architecture & Design 7
   3.1. High-Level Architecture Overview 7
   3.2. Component Descriptions 7
   3.3. BLE/GPS Handshake Process 8
   3.4. Workflow Descriptions 8
   3.5. Design Constraints 9
   3.6. Security Considerations 9
4. Data Requirements 10
   4.1. Data Types Stored 10
   4.2. Data Format 10
   4.3. Data Flow Summary 10
5. Implementation 11
   5.1. Development Approach 11
   5.2. ESP32 Setup and Upload Issues 11
   5.3. ESP32 Hardware Validation (LED Test) 13
   5.4. ESP32 Serial Communication Setup 14
   5.5. GPS Integration Attempt 14
   5.6. GPS Debugging Process 16

5.6.1 Library and Software Configuration 16
5.6.2 Baud Rate Testing 17
5.6.3 Wiring Verification 17
5.6.4 System Isolation and Minimal Testing 17
5.6.5 Serial Monitor Configuration 18
5.7. BLE Integration Attempt 18
5.8. BLE Data Handling 18
6. Testing & Evaluation 20
6.1. GPS Testing Results 20
6.2. GPS Evaluation 20
6.3. BLE Testing Results 20
6.4. BLE Evaluation 21
6.5. BLE Workaround and Validation 22
7. Conclusion 23
References: i
Appendix A: Report Structure Example ii
Appendix B: Code Listing / GitHub Project Link iv

Table of Figures

FIGURE 1 ATLANTIC TECHNOLOGICAL UNIVERSITY DONEGAL LOGO**ERROR! BOOKMARK NOT DEFINED.**

FIGURE 2 ATU DONEGAL, LETTERKENNY.....**ERROR! BOOKMARK NOT DEFINED.**

Glossary of Terms

Term	Definition
BLE (Bluetooth Low Energy)	A type of wireless communication designed for short-range, low-power data exchange that operates in the 2.4 GHz band. Commonly used for proximity sensing, wearables, and IoT devices where battery efficiency is important. (*Fürst et al., 2018*)
GPS (Global Positioning System)	A satellite-based navigation system that provides absolute location and accurate time information anywhere on Earth. Alternative versions such as GLONASS (Russian Federation), BeiDou (China) and Galileo (European Union) also exist, but they all perform the same function. (*Rutkowski, 2022*)
RSSI (Received Signal Strength Indicator)	A measurement of how strong a received radio signal is. In BLE systems, RSSI is used to estimate how near or far a device is. (*Chai, An and Du, 2016*)
ESP32	A microcontroller unit with integrated Wi-Fi and Bluetooth connectivity for a wide-range of applications. (*ESP32 Wi-Fi & Bluetooth SoC | Espressif Systems, no date)*
NEO-8M	A small GPS receiver module that outputs location data using standard protocols like NMEA. It connects to microcontrollers via UART and provides reliable GPS fixes. It is a specific type of GNSS module. (*NEO-M8 series, 2015*)
GNSS	A Global Navigation Satellite System (GNSS) module is an electronic device that receives signals from satellite constellations such as GPS, GLONASS, Galileo, or BeiDou to calculate its geographic position in real time. (*Rutkowski, 2022)*
NMEA (National Marine Electronics Association Format)	A standardised text-based format used by GPS modules to transmit location, time, and status data. Messages are structured as comma-separated sentences starting with a "$" symbol. (*Gakstatter, 2015*)

ms (milliseconds)	A unit of time equal to one-thousandth of a second (0.001 s). (*Merriam-Webster, 2025*)
Hz (Hertz)	A unit of frequency meaning equal to one cycle per second. (*Merriam-Webster, 2025*)
GGA	An NMEA sentence that provides key GPS fix information, including latitude, longitude, fix quality, and number of satellites. Used in this project to validate the reliability of each GPS reading.
UART (Universal Asynchronous Receiver-Transmitter)	A commonly used asynchronous serial communication protocol. It is the protocol used by Arduino boards and computers in order to communicate.

1. Project Overview

1.1. Purpose

I will investigate how Bluetooth Low Energy (BLE) and Global Positioning System (GPS) technologies can be combined to create a hybrid 'sensing' model capable of supporting context-aware IoT applications to determine location. My primary goal will be to design and evaluate a BLE/GPS 'handshake mechanism', where BLE proximity events and GPS location updates interact to determine meaningful system states. I will focus on the technical synchronisation between BLE's short-range RSSI-based detection and GPS's global positional data, examining how these two sensing modalities can be used together to deliver reliable location tagging on a low-power microcontroller platform. I am hoping to provide a clear understanding of the limitations, timing behaviours, and accuracy characteristics involved in hybrid BLE/GPS sensing.

1.2. Objectives

1.2.1 BLE Proximity Detection

Implement a BLE 4.2 scanning system on the ESP32 that detects a predefined BLE tag, retrieves its RSSI values, and classifies proximity states (e.g., near, far, lost). I chose BLE 4.2 because it is already integrated into the ESP32 hardware (Espressif, 2025). This means no additional antennas, modules, or power overhead are required. It is more than capable of delivering the short-range proximity readings I need, and it maintains low power consumption while still offering stable scanning performance. The system must identify 'tag-loss' events within a scanning interval of <500 ms (Bluetooth SIG, 2014) and operate reliably across typical indoor and outdoor environments.

1.2.2 GPS Location Detection/Acquisition

Integrate the NEO-8M GPS module to obtain real-time coordinates. GPS fix-quality and satellite-count data will be used to validate readings and prevent false classifications. (*NEO-M8 series*, 2015)

1.2.3 BLE/GPS Handshake

Design a deterministic "handshake" workflow where BLE proximity loss triggers GPS location verification, and GPS updates influence BLE scanning behaviour. The handshake must define clear timing rules, thresholds, and decision logic for transitions between contextual system states (e.g., departure detected, geofence entry, false BLE loss).

1.2.4 Event Logging

Develop a local logging system using internal flash of the ESP32 or MicroSD storage to record BLE RSSI readings, GPS coordinates, fix quality, handshake outcomes, and system states.

1.2.5 System Feedback

Implement a visual or auditory output mechanism that reflects handshake-determined system states such as "BLE tag lost", "geofence entered", or "GPS invalid". Feedback must be driven solely by validated BLE/GPS transitions.

1.2.6 Performance Evaluation

Evaluate the hybrid sensing system by analysing BLE range variability, GPS lock time, handshake timing behaviour, and false-positive/false-negative rates using the collected log data. This evaluation will determine the reliability and limitations of the coordinated BLE/GPS sensing model.

1.3. Project Scope

In Scope:

- Development of a hybrid BLE/GPS sensing system on the ESP32 platform.
- Implementation of BLE scanning, GPS location/data acquisitions, handshake logic and event logging
- Technical evaluation of synchronisation timing, accuracy, and system reliability.

Out of Scope:

- Cloud integration, remote databases, or mobile app development.
- Behavioural user testing or human-factor evaluation.
- AI-based modelling, multi-user capabilities, or commercial product design.
- Environmental/social impact analysis.

Assumptions:

- Adequate GPS satellite visibility during operation.
- BLE tags comply with Bluetooth 4.2 advertising behaviour. Components are often branded as being compliant when they, in fact, are not.

- Local power (vehicle or battery) is available.
- Environmental conditions permit stable BLE and GPS communication.

Dependencies:

- ESP32 BLE and GPS software libraries.
- ESP32's BLE version correctly branded.
- Reliable NEO-8M GPS operation and NMEA sentence output.

2. Requirements

2.1. Functional Requirements

BLE Scanning

- The system shall scan for a specific BLE tag at an interval of <500 ms.
- The system shall retrieve RSSI values and classify proximity states.

Proximity Loss Detection

- The system shall determine tag loss after X consecutive missed scan cycles. (X shall be determined at a later date).

GPS Coordinate Acquisition

- The system shall retrieve GPS coordinates at a minimum of 1 Hz. (*NEO-M8 series*, 2015)
- The system shall validate readings using fix quality and satellite count. (Active Research Ltd., 2015)

Geofence Detection

- The system shall detect when coordinates enter or exit predefined geofences.

BLE/GPS Handshake

- The system shall trigger GPS checks when BLE proximity loss occurs.
- The handshake shall classify transitions such as departure, arrival, and false loss.

Event Logging

- Logs shall include timestamps, RSSI, GPS data, handshake results, and state transitions.

Feedback Output

- The system shall output visual/auditory indicators for state changes.

2.2. Non-Functional Requirements

Performance

- The system shall maintain a BLE scan interval of <500 ms and retrieve GPS coordinates at a minimum frequency of 1 Hz to ensure a responsive BLE/GPS handshake execution.

Accuracy

- The system shall achieve a typical GPS positional accuracy of 2.5 - 5 metres in open-sky conditions (*NEO-M8 series*, 2015).

Reliability

- The system shall operate continuously without unexpected interruptions to BLE scanning or GPS data acquisition, and it shall recover quickly from temporary GPS signal loss.

Robustness to Environmental Factors (Ireland)

- The system shall remain functional under moderate environmental interference such as BLE multipath effects, signal attenuation, and varying satellite visibility due to adverse weather conditions.

Power Efficiency

- The system should be capable of operating on portable power for at least 2 hours, with BLE scan intervals and GPS polling rates chosen to balance responsiveness and power usage.

Security

- The system should avoid the storage of personal or identifying data and should use BLE advertising mode without pairing to reduce security risks.

3. System Architecture & Design

3.1. High-Level Architecture Overview

My system will be built around a hybrid sensing approach that combines BLE for short-range proximity detection with GPS for absolute, accurate positional awareness. The ESP32 microcontroller acts as the central controller, coordinating the built-in BLE 4.2 and GPS subsystems and executing the handshake logic that determines the system's context. The overall architecture is deliberately simple at this stage, as the detailed implementation will be carried out next semester.

The key components of the architecture are:

- BLE subsystem, which handles scanning for a specific tag and reading RSSI values; BLE scans occur at <500 ms intervals
- The GPS subsystem receives positional data from the NEO-8M module using NMEA sentences (Active Research Ltd., 2015) and updates at approximately 1 Hz.
- Handshake engine, which evaluates BLE and GPS data together to determine system state transitions.
- Dual sensor layer, which compares proximity information with location information.
- Event logger, which records system events and environmental data to either SD card or internal flash.

This high-level structure outlines how the major parts of the system are intended to work together, without going into implementation detail. This will be investigated and fine-tuned next semester.

3.2. Component Descriptions

Bluetooth Low Energy Module

BLE is used to detect the presence or absence of a designated tag by scanning for its advertising packets and reading the associated RSSI values. The ESP32's built-in BLE 4.2 support makes it suitable for short-range proximity detection without requiring additional hardware. BLE range and signal strength vary depending on the environment, and RSSI readings may fluctuate due to

interference or obstacles. The scan interval determines how quickly the system can detect proximity changes and affects overall power usage.

GPS Module (NEO-8M)

The NEO-8M GPS module provides global position data at an update rate of approximately one hertz, with typical accuracy of around 2.5 metres under good satellite visibility. Cold-start times range from roughly twenty to thirty seconds. The module outputs NMEA sentences such as GGA, which provide coordinates, fix quality, and satellite count. GPS performance reduces significantly indoors or in areas with obstructed sky view.

3.3. BLE/GPS Handshake Process

The handshake between BLE and GPS is the central behaviour that I will investigate. When the BLE subsystem detects that the tag's RSSI has fallen below a threshold or that the tag has not appeared for a predefined number of scans, it flags a potential loss event. This triggers the handshake engine to request an updated GPS reading.

The GPS subsystem returns positional data along with fix quality and satellite count, allowing the system to assess whether the reading is reliable. Based on the combined BLE and GPS information, the handshake engine classifies the situation.
Examples include:

- the tag has genuinely moved out of range and the GPS position indicates that the user has left the area
- the BLE signal dropped due to interference, but the GPS position indicates the user is still within the expected area
- the GPS data is unreliable due to poor fix, in which case the event is ignored or deferred.

The handshake outcome leads to an internal state transition and the creation of a log entry

3.4. Workflow Descriptions

In a departure scenario, the BLE signal is initially strong. As the user moves away, RSSI decreases until the tag is no longer detected. This will initiate an internal state transition, which will then trigger the handshake. If GPS confirms the user has left the defined geofence, the system records

this as a departure event. In an arrival scenario, the device receives periodic GPS updates and detects that the coordinates fall within a predefined location. The system updates its state accordingly. If the BLE tag is still missing at this point, the system may classify this as an arrival without the tagged item.

3.5. Design Constraints

Several constraints are expected at this stage. BLE RSSI is inherently noisy (Lee *et al.*, 2025) meaning proximity readings may fluctuate even when the tag has not moved. GPS accuracy varies with weather conditions, satellite geometry, and environmental obstructions. The ESP32 must balance BLE scanning and GPS parsing within its available processing resources. Power consumption may increase if the scan interval or GPS polling frequency is set too aggressively. These are variables that will become evident when I begin building the artefact.

3.6. Security Considerations

The system does not collect personal information and does not use any cloud or network components. BLE operates in unpaired advertising mode, which limits potential security concerns. Logged data consists only of device identifiers and environmental readings taken from the NMEA sentence.

4. Data Requirements

4.1. Data Types Stored

The system will store the following data to support analysis and evaluation:

- GPS latitude and longitude, representing the device's absolute position
- GPS fix quality and satellite count, used to assess the reliability of each coordinate reading
- BLE RSSI values, representing the detected proximity of the tagged device
- Presence or absence of the BLE tag, derived from the BLE scan results
- Internal state transitions, generated by the BLE/GPS handshake logic

4.2. Data Format

The data will be stored in a simple, structured format suitable for later analysis. The following is both appropriate and readable:

lat, lon, fixQuality, satCount, rssi, eventType

This approach is human readable and allows the data to be imported easily into spreadsheets or analysis tools if it's saved as a CSV file.

4.3. Data Flow Summary

- BLE scanning produces RSSI values and presence information.
- GPS outputs positional data through NMEA sentences.
- The handshake engine evaluates both data sources and determines a classification.
- The logger records all relevant information to local storage.

5. Implementation

5.1. Development Approach

The system was built incrementally using the ESP32-2432S028R development board, the AT-09 Bluetooth 4.0 module, and the u-blox NEO-8M GPS module. Development was carried out using the Arduino IDE.

Rather than attempting to implement the entire system at once, each component was built and tested as independently as possible. This allowed potential problems to be isolated early and reduced complexity during debugging.

The implementation process followed a structured sequence:

1. Verifying basic hardware functionality
2. Establishing communication with the ESP32
3. Implementing serial communication for debugging
4. Attempting GPS communication via UART
5. Integrating BLE communication
6. Developing core system logic

This iterative/prototyping development model ensured that issues could be traced back to specific components rather than the system as a whole.

5.2. ESP32 Setup and Upload Issues

The ESP32 was configured using the "ESP32 Dev Module" profile within the Arduino IDE. Scripts of code in Arduino IDE are called sketches, and initial sketch uploads failed with repeated connection errors, specifically a serial timeout during upload.

Blink_Script.ino

```
// LED in GPIO pin 4
#define LED 4

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  // Blink LED light on for 1 second (1000)
  digitalWrite(LED, HIGH);
  delay(1000);
  // Blink LED off for 1 second
  digitalWrite(LED, LOW);
  delay(1000);
}
```

Output

```
Sketch uses 269688 bytes (20%) of program storage space. Maximum is 1310720 bytes.
Global variables use 22060 bytes (6%) of dynamic memory, leaving 305620 bytes for local variables. Maximum is 327680 bytes.
esptool v5.1.0
Serial port COM6:
Connecting...
A serial exception error occurred: Write timeout

Note: This error originates from pySerial. It is likely not a problem with esptool, but with the hardware connection or drivers.
For troubleshooting steps visit: https://docs.espressif.com/projects/esptool/en/latest/troubleshooting.html
Failed uploading: uploading error: exit status 1
```

Figure 1: Serial exception error

This issue was traced to incorrect port selection and the ESP32 not entering bootloader mode. The correct port was identified by observing changes in available COM ports when the device was connected. The board was then manually placed into bootloader mode using the "BOOT" button during upload. After correcting these issues, sketches uploaded successfully and it worked as expected.

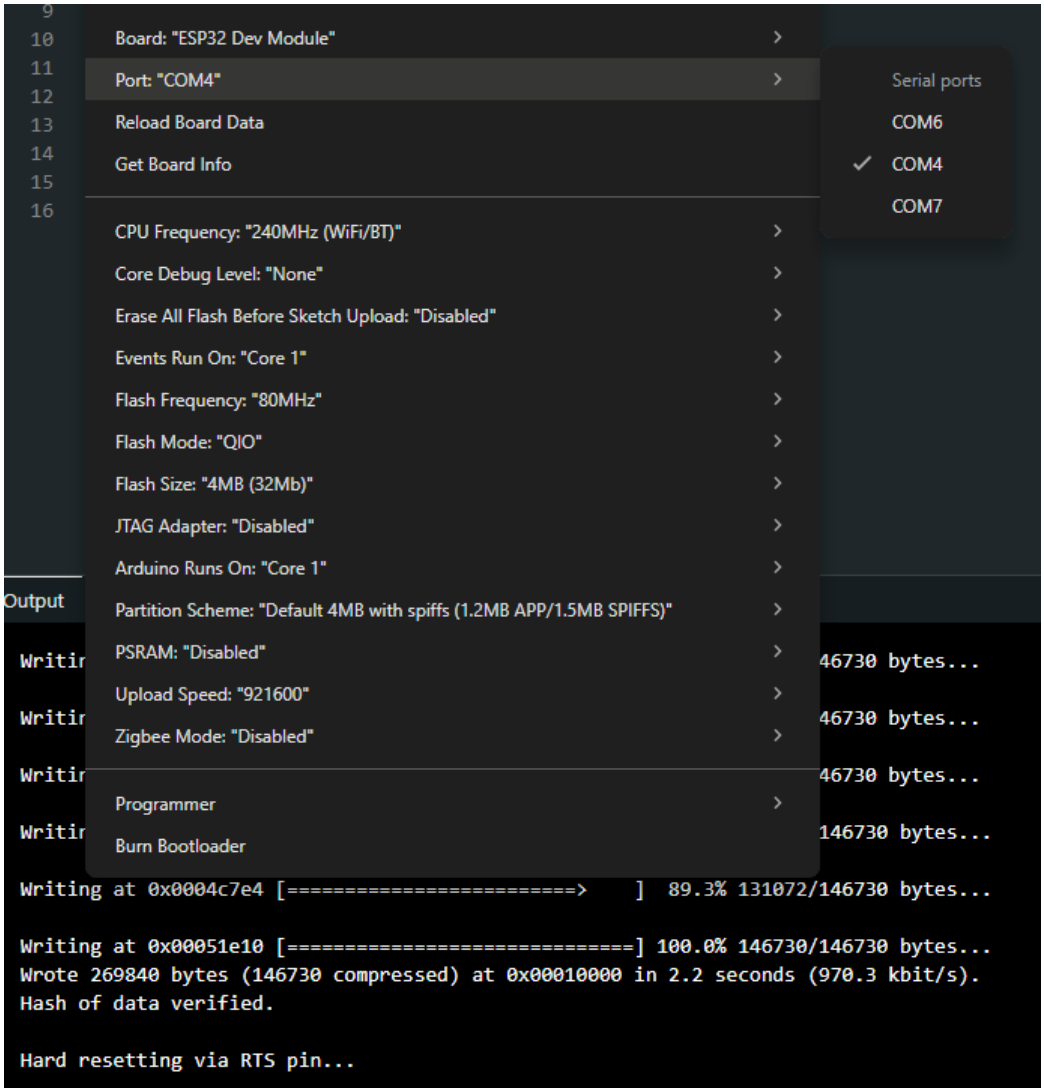

Figure 2: Correct serial port selected & upload log successfully uploaded

5.3. ESP32 Hardware Validation (LED Test)

A simple LED blink test was used to confirm that the ESP32 was functioning correctly. The LED connected to GPIO4 successfully toggled on and off using the built-in "blink" sketch in the IDE.

This confirmed that:

1. Communication between the ESP32 and the computer was successful
2. Code was being uploaded, executed and was compatible with the ESP32 microcontroller
3. The GPIO pins on the board were functioning as expected

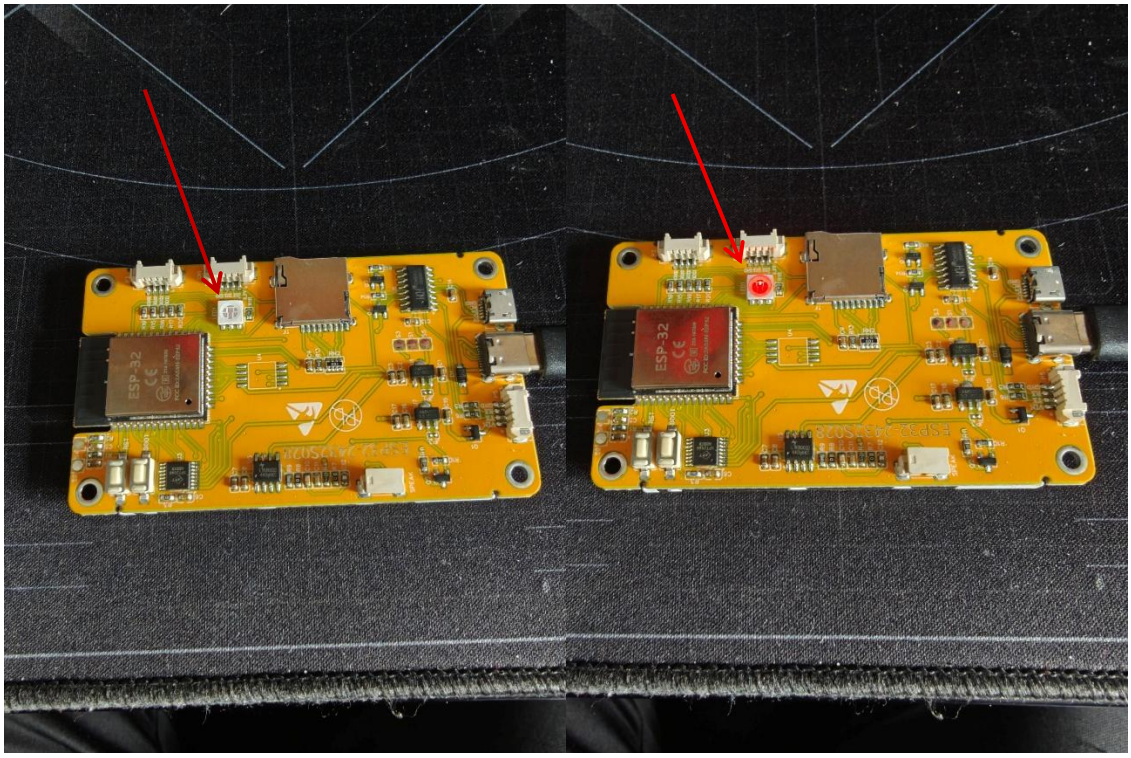

Figure 3:LED light toggling off/on

5.4. ESP32 Serial Communication Setup

Serial communication was then implemented to allow debugging in real time. Messages were printed from the ESP32 to the Serial Monitor, and the output appeared correctly. This confirmed that USB communication between the ESP32 and development environment was functioning as expected and that the Serial Monitor within Arduino IDE could be used for further diagnostics if required.

5.5. GPS Integration Attempt

The GPS module was connected to the ESP32 via UART using the TX and RX pins. A minimal test program was written to read incoming data from the GPS module and print it directly to the Serial Monitor.

The expected output was standard NMEA sentences, which include information such as latitude, longitude, GPS fix quality, and the number of connected satellites.

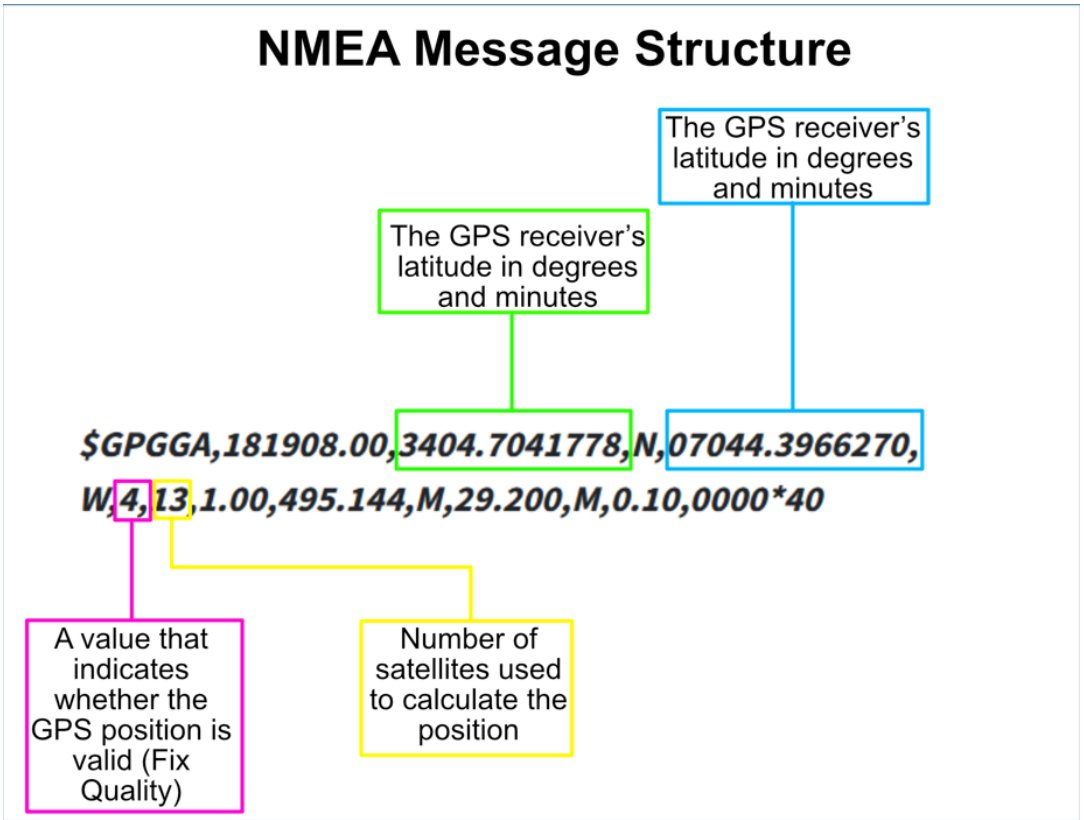

Figure 4:Format of expected NMEA sentence output

However, when the test was carried out, the output appeared as unreadable or garbled characters instead of valid NMEA sentences.

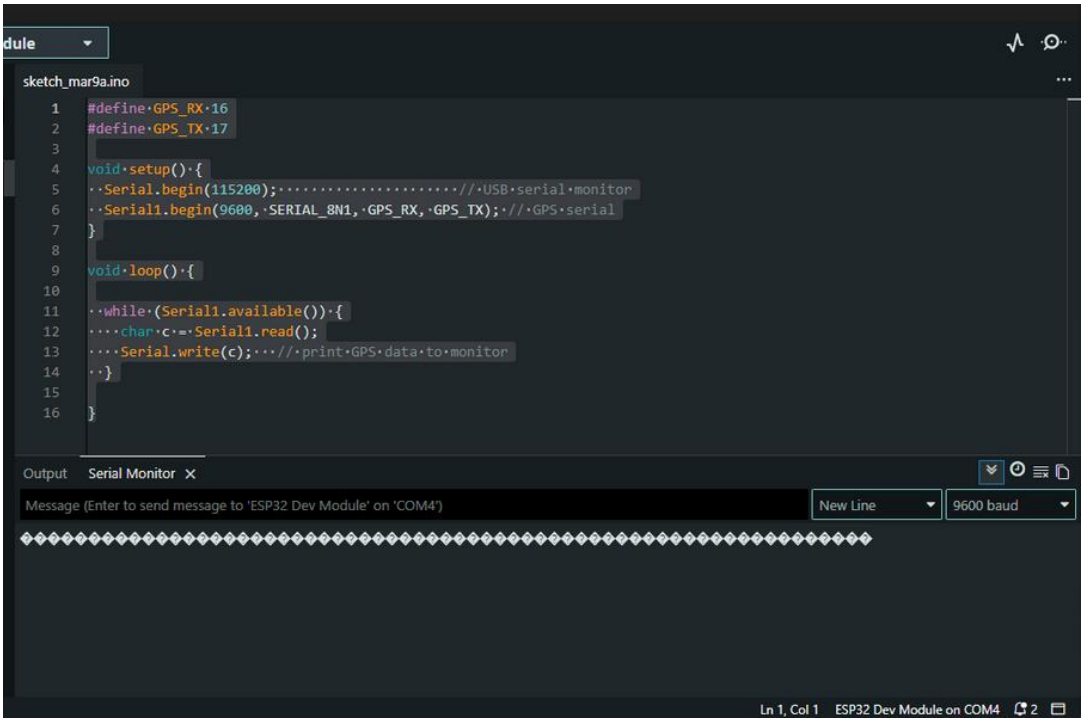

Figure 5:Received garbled output from GPS module

5.6. GPS Debugging Process

A structured and iterative debugging process was carried out to identify the cause of the incorrect GPS output. Since some form of data was consistently being received, the issue was treated as a communication or configuration problem rather than a complete hardware failure, therefore, multiple potential causes needed to be investigated.

5.6.1 Library and Software Configuration

Multiple Arduino libraries were tested to determine whether the issue was related to how the software was interpreting the data. Display related libraries such as Adafruit GFX, Adafruit SSD1306, and TFT_eSPI, were initially used to test whether the issue was limited to the Serial Monitor output. The IDE was still receiving the same garbled output, and the libraries were unable to display anything on the ESP32 OLED screen as anticipated. These libraries did not resolve the issue, indicating that the problem was not related to display handling.

GPS specific libraries, including TinyGPS, TinyGPSPlus, NeoGPS, and the SparkFun u-blox GNSS library, were then tested. These libraries are designed to parse NMEA data into usable values. However, all of them require valid input data. Since the incoming data was already corrupted and/or in an unknown format, none of the libraries were able to produce meaningful

output. This confirmed that the issue existed before the parsing stage and was likely related to the data being received rather than how it was processed.

5.6.2 Baud Rate Testing

Since UART communication depends on matching baud rates between devices, multiple standard GPS baud rates were tested, including 4800, 9600, 38400, and 115200.

Each configuration was uploaded to the ESP32, and the Serial Monitor was checked for valid output. Despite testing these common baud rates, the output remained uninterpretable.

5.6.3 Wiring Verification

All wiring connections were rechecked to ensure correct configuration. The TX and RX connections were verified, and power and ground connections were confirmed to be stable and working. The GPS RX pin was considered non-essential and was temporarily disconnected to reduce possible interference. Multiple GPS modules were also tested to rule out a faulty unit.

A flashing blue LED on the GPS module indicated that it was powered and actively receiving satellite data.

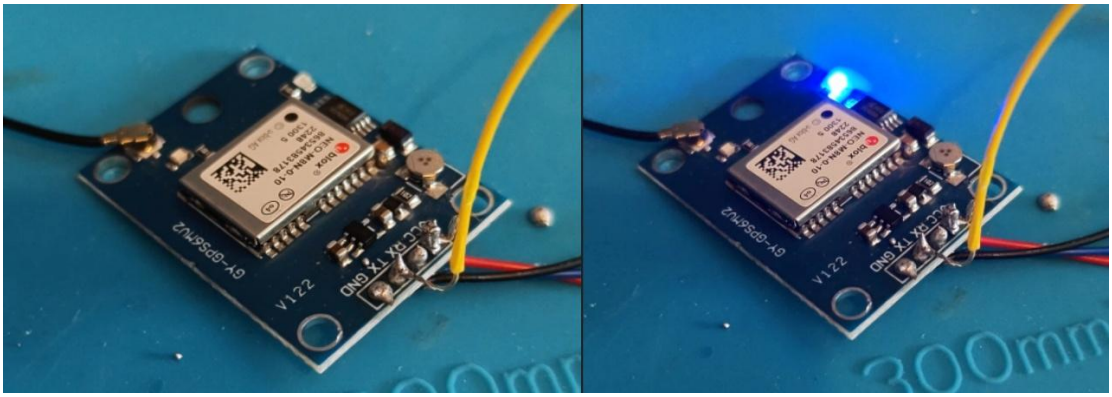

Figure 6: Blue flashing light indicating GPS data is being received.

5.6.4 System Isolation and Minimal Testing

To eliminate interference from other components or software complexity, all non-essential hardware connections were removed and a minimal test sketch was used, which only read raw serial data from the GPS module.

This ensured that the ESP32 was performing only a single task, with no additional libraries or processes that could affect device timing or communication. Despite this controlled setup, the output remained unchanged.

5.6.5 Serial Monitor Configuration

The Serial Monitor configuration was checked and verified to ensure correct interpretation of incoming data. The baud rate was matched to the code, and alternative baud rates were also briefly tested. No changes in output were observed, confirming that the issue was not caused by Serial Monitor settings.

5.7. BLE Integration Attempt

An external AT-09 Bluetooth 4.0 module was used to simulate a BLE tag device. Although the ESP32 itself includes built-in BLE functionality, the external module allowed for separation between scanning and broadcasting devices. The module was connected via UART and configured using AT commands within the IDE serial monitor. Communication between the ESP32 and the module was successful, with the module working as expected. The module was able to broadcast and be detected by other devices, confirming that BLE functionality was operational.

5.8. BLE Data Handling

When attempting to read data from the BLE module, the output appeared as garbled characters (similar to the results of the GPS module). After further investigation, it was determined that the module was outputting raw binary data rather than ASCII text.

To resolve this, the data was read byte-by-byte and converted into hexadecimal format before being printed to the Serial Monitor. This allowed the data to be visualised in a structured and readable way.

BLuetooth_Garbled.ino

```
#define RXD2 16
#define TXD2 17

HardwareSerial AT09(2); // UART2

void setup() {
  Serial.begin(115200);
  while(!Serial);

  Serial.println("=== AT-09 BLE Test ===");
  AT09.begin(9600, SERIAL_8N1, RXD2, TXD2);
}

void loop() {
  // Print any data from AT-09 to Serial Monitor
  while(AT09.available()) {
    char c = AT09.read();
    Serial.write(c);
  }
}
```

Output Serial Monitor ×

Message (Enter to send message to 'ESP32 Dev Module' on 'COM4')

Figure 7: Garbled output from BLE module

6. Testing & Evaluation

6.1. GPS Testing Results

The GPS module was tested using a minimal serial read program. The expected result was the output of valid NMEA sentences (Figure 4). However, the actual output consisted entirely of unreadable characters (Figure 5).

After successfully converting the garbled output from the BLE to hexadecimal format and then into standard characters, the same process was carried out with the GPS, however, the results were ultimately unsuccessful.

6.2. GPS Evaluation

Although valid NMEA data was not obtained, consistent data transmission was observed from the GPS module. This confirms that the GPS module was powered and actively sending and receiving data, and that a communication channel existed between the module and the ESP32.

The failure to produce readable output suggested a low-level communication issue, such as unknown or incorrect baud rate configuration, incompatible signal levels, or non-standard data formatting.

All standard debugging procedures were followed, including library testing, wiring verification, baud rate adjustment, and system isolation. Since none of these resolved the issue, the problem is likely due to hardware compatibility or signal-level mismatch rather than software implementation.

This type of issue occurs below the level of standard software debugging and would not have been easily predicted during the design phase as it is not mentioned in any standard device documentation.

6.3. BLE Testing Results

The BLE module responded correctly to AT commands and was able to broadcast as expected. However, when reading data directly, the output initially appeared as garbled characters (Figure 7).

6.4. BLE Evaluation

Further investigation showed that it was not an encoding issues, but that the BLE module was outputting raw binary data. Once converted into hexadecimal format, the data became readable, confirming that communication was functioning correctly.

BLE_Hex.ino

```
  // Start UART2 for the AT-09 BLE module
  // 9600 is the default baud rate for AT-09
  // SERIAL_8N1 = standard config (8 data bits, no parity, 1 stop bit)
  // RXD2, TXD2 is the GPIO pins defined above
  AT09.begin(9600, SERIAL_8N1, RXD2, TXD2);

  Serial.println("AT-09 BLE raw data monitor");
}

void loop() {
  // Check if there's any data coming from the BLE module
  while (AT09.available()) {

    // Read one byte from UART2
    uint8_t b = AT09.read();

    // If the value is less than 16, it means it's a single HEX digit
    //add a leading 0 so formatting stays consisten (like 0A instead of A)
    if (b < 16) Serial.print("0");

    // Print the byte in HEX format
    Serial.print(b, HEX);

    // Add a space between each byte for readability
    Serial.print(" ");
  }
}
```

Output Serial Monitor ×

Message (Enter to send message to 'ESP32 Dev Module' on 'COM4')

```
DF A6 08 C4 D1 00 00 00 00 00 00 40 00 C0 F8 FF FE FE FE FE FE FE FE FC 6E 18 7C F6
```

Look_For_Bluetooth_Module.ino

```
#include <BLEDevice.h>
#include <BLEScan.h>

BLEScan* pBLEScan;

void setup() {
  Serial.begin(115200);
  BLEDevice::init("");

  pBLEScan = BLEDevice::getScan();
  pBLEScan->setActiveScan(true);
}

void loop() {

  Serial.println("=== Devices Nearby ===");

  BLEScanResults* results = pBLEScan->start(3, false);

  for (int i = 0; i < results->getCount(); i++) {

    BLEAdvertisedDevice device = results->getDevice(i);

    Serial.print("Device: ");
    Serial.print(device.getAddress().toString().c_str());

    Serial.print("  RSSI: ");
    Serial.println(device.getRSSI());
  }


  delay(8000);
}
```

Serial Monitor × Output

Message (Enter to send message to 'ESP32 Dev Module' on 'COM4')

```
Device: 28:6b:b4:ae:33:e2  RSSI: -90
Device: 2e:9c:76:3a:49:8b  RSSI: -78
Device: 32:82:8a:53:c6:55  RSSI: -50
Device: 39:1a:78:da:79:a9  RSSI: -75
Device: 6d:28:d9:a7:81:c0  RSSI: -98
Device: 8c:ea:48:55:6a:bf  RSSI: -96
Device: d3:2d:42:46:17:38  RSSI: -100
```

Figure 8: Output being converted to HEX (left) and then into readable characters (right)

During the process of extended testing of the BLE module, a further more significant issue was discovered. The MAC address of the BLE module would change every few minutes which would lead to a lost signal. The module would then reappear under a different MAC address.

```
Scanning for BLE devices...
MAC: 00:49:2a:fb:eb:e4 | Name:  | RSSI: -32
MAC: 23:c3:4e:f1:87:b8 | Name:  | RSSI: -62
MAC: 25:1f:ff:dc:44:bf | Name:  | RSSI: -98
MAC: 28:6b:b4:ae:33:e2 | Name: Fridge | RSSI: -87
MAC: 28:ed:aa:e4:ee:43 | Name:  | RSSI: -57
MAC: 32:ab:c2:29:9f:90 | Name: Samsung_S20_FE  | RSSI: -57
MAC: 4b:32:d7:c9:96:17 | Name: Garmin_Forerunner_245_Music | RSSI: -44
MAC: 75:a8:2b:0a:8a:a3 | Name:  | RSSI: -101
MAC: 76:de:13:15:18:a9 | Name:  | RSSI: -86
MAC: d3:2d:42:46:17:38 | Name: Govee_H6099_1738 | RSSI: -97
MAC: d6:79:6b:09:26:e4 | Name:  | RSSI: -97
MAC: f0:83:63:d1:ce:7a | Name:  | RSSI: -103
------ SCAN END ------
```

```
Scanning for BLE devices...
MAC: 00:46:8c:ff:bb:a2 | Name:  | RSSI: -36
MAC: 23:c3:4e:f1:87:b8 | Name:  | RSSI: -61
MAC: 25:1f:ff:dc:44:bf | Name:  | RSSI: -90
MAC: 28:6b:b4:ae:33:e2 | Name: Fridge | RSSI: -85
MAC: 28:ed:aa:e4:ee:43 | Name:  | RSSI: -50
MAC: 32:ab:c2:29:9f:90 | Name: Samsung_S20_FE  | RSSI: -57
MAC: 4b:32:d7:c9:96:17 | Name: Garmin_Forerunner_245_Music | RSSI: -44
MAC: 75:a8:2b:0a:8a:a3 | Name:  | RSSI: -95
MAC: 76:de:13:15:18:a9 | Name:  | RSSI: -86
MAC: d3:2d:42:46:17:38 | Name: Govee_H6099_1738 | RSSI: -97
MAC: d6:79:6b:09:26:e4 | Name:  | RSSI: -97
MAC: f0:83:63:d1:ce:7a | Name:  | RSSI: -100
------ SCAN END ------
```

Figure 9: BLE's MAC address changing between scans

This behaviour prevented reliable identification of the device, making it unsuitable for consistent proximity tracking. While the BLE communication itself was successful, the instability of the module limited its practical use.

6.5. BLE Workaround and Validation

Therefore, to determine whether the issue was being caused by the ESP32 or the BLE module, a controlled test was implemented using a predefined identifier. The ESP32 was programmed to detect a known device and process its RSSI values. Under these conditions, the system functioned correctly, detecting the device and calculating proximity as expected.

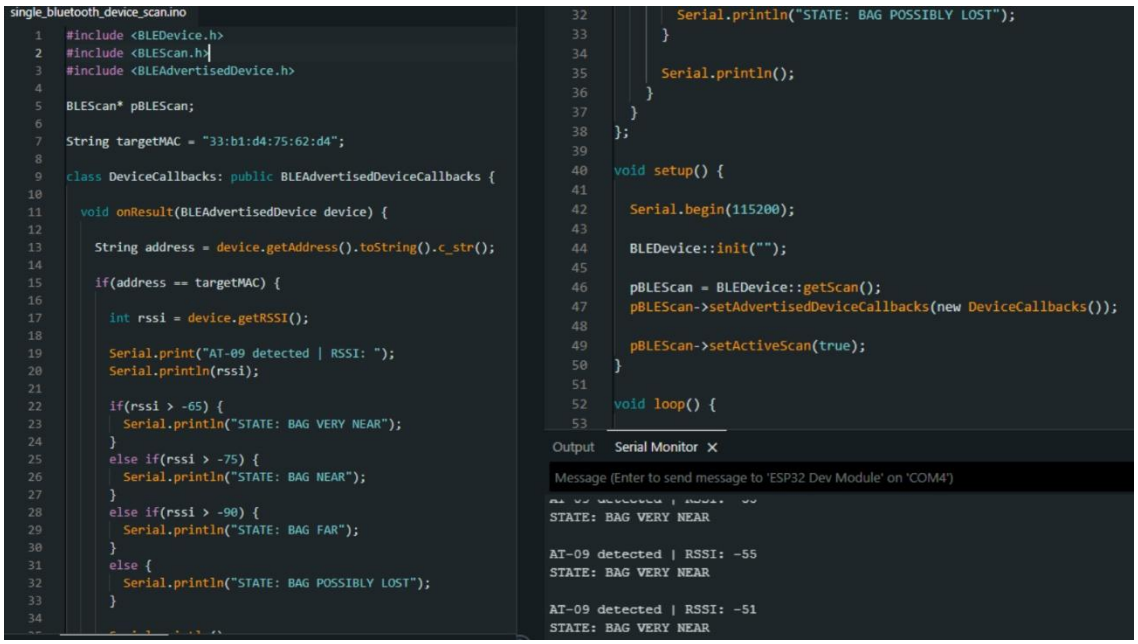

Figure 10: Proximity detection working as designed

This confirmed that BLE scanning functionality was working correctly, RSSI-based proximity detection was valid, and the issue of the changing MAC address was isolated to the BLE module hardware itself. This represents a partial success, as the system logic was validated despite hardware limitations.

Due to the nature of the cheap component, hard-coding a MAC address onto it was not possible. After further investigation into the device, it was assumed that the changing MAC address was a preconfigured security feature built into the firmware of the BLE module that could not be altered or removed.

7. Conclusion

The overarching aim of this project was to investigate how effectively Bluetooth Low Energy and GPS could work together to provide seamless location tagging.

The ESP32 platform was successfully configured and used to implement serial communication and act as a core system bridge to external components.

The BLE subsystem achieved partial success, with communication established and the detection logic validated under controlled conditions using predefined values. However, limitations in the AT-09 module, particularly MAC address instability, prevented consistent real-world tracking.

The GPS subsystem did not produce valid NMEA output, although consistent data reception indicated that communication was active. This suggests a low-level compatibility or data interpretation issue rather than a complete failure.

Despite full system integration not being achieved, key aspects of the system were successfully demonstrated. The BLE detection logic and system design were validated, and the challenges encountered provided useful insight into real world hardware limitations.

References:

Appendix A: Report Structure Example

<u>Project Technical Report</u>

1) Project Overview

- Purpose and objectives of the software
- Scope of the project
- Assumptions, constraints and dependencies
- Definitions and acronyms

2) Requirements

Functional Requirements

- Detailed description of system functions and capabilities
- Input and output specifications
- Data processing operations
- Use cases, user stories (if applicable)

Non-Functional Requirements

- Performance requirements (e.g., response time, throughput)
- Security requirements
- Usability and accessibility standards
- Reliability and availability metrics
- Scalability considerations *(if applicable)*

3) System Architecture & Design

- Hardware and software requirements (including development and runtime environment)
- High-level system architecture
- Key components/functions/objects and their interactions
- Detailed description of system features, including workflow descriptions.
- Design and implementation constraints
- System qualities (e.g., maintainability, extensibility)
- Security protocols / considerations
- Testing strategy

4) Data Requirements

- Data models and schemas
- Data flow diagrams, class diagram or similar

5) Implementation

- Document the development process
- Code management, version control, etc. *(evidence of following best practice)*
- Discuss challenges, errors and limitations

6) Testing & Evaluation

- Testing overview / discussing testing requirements
- Functional Testing / Analysis *(following testing strategy)*
- Results/Evaluation Discussion

7) Conclusion

- Project conclusion/findings
- Limitations
- Future Work

Appendix B: Code Listing / GitHub Project Link